VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

Karan Suresh Nair (1BM23CS139)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Aug-2025 to Dec-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled " Bio Inspired Systems (23CS5BSBIS)" carried out by **Karan Suresh Nair (1BM23CS139),** who is a bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Dr. Naresh E Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Github Link:
https://github.com/KaranSureshNair/BIS_LAB_139

USN: 1BM23CS139

INDEX

Name KARAN SURESH NAIR

Standard 5 Section C Roll No. 139

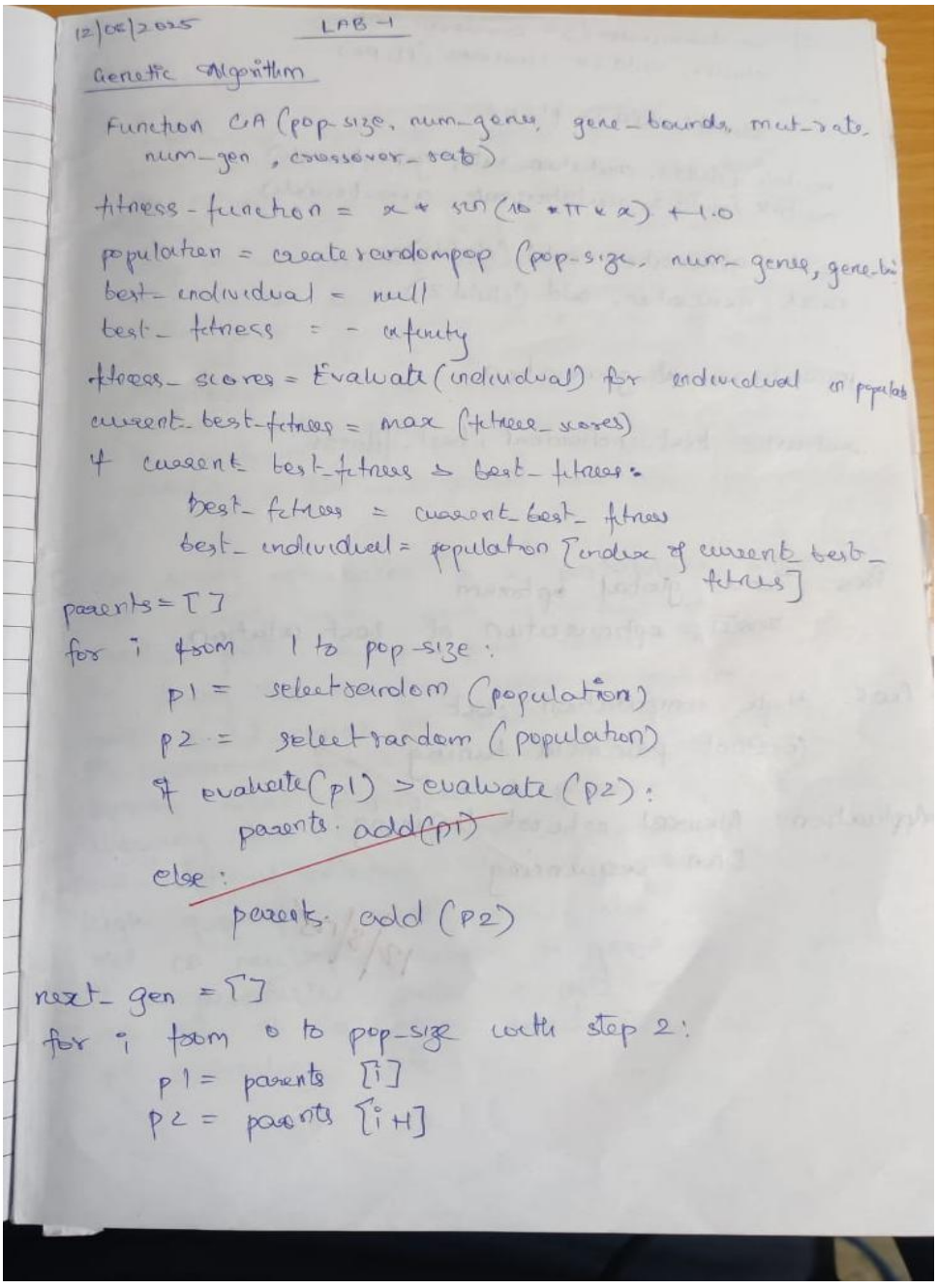
Subject BIS LAB

SL No.	Date	Title	Page No.	Teacher Sign / Remarks
1	12/08	LAB-1		19/8/25
2	19/08	LAB-2		
3	28/09	Particle Swarm Optimisation		
		Optimisation of Gene Expression		9/9/25
4	16/09	Ant colony	10/10	
5	25/9	Cuckoo search Optimisation	10/10	23/9/2
6	14/10	Grey Wolf Optimiser	10/10	14/10/2
7	28/10	Parallel cellular Algorithm.	10/10	28/10/25
		10/10		11/11/25

Program 1

Design and implement a **genetic algorithm** in Python that evolves a population of random character strings to match a given target phrase ("HELLO WORLD").

Algorithm:



12/08/2025 LAB-1

Genetic Algorithm

```
Function GA(pop_size, num_genes, gene_bounds, mut_rate,
    num_gen, crossover_rate)
fitness_function = x * sin(10 * π * x) + 1.0
population = createrandompop(pop_size, num_genes, gene_bo
best_individual = null
best_fitness = -infinity
fitness_scores = Evaluate(individual) for individual in popula
current_best_fitness = max(fitness_scores)
if current best_fitness > best_fitness:
    best_fitness = current_best_fitness
    best_individual = population[index of current_best_
                                 fitness]
parents = []
for i from 1 to pop_size:
    p1 = selectrandom(population)
    p2 = selectrandom(population)
    if evaluate(p1) > evaluate(p2):
        parents.add(p1)
    else:
        parents.add(p2)
next_gen = []
for i from 0 to pop_size with step 2:
    p1 = parents[i]
    p2 = parents[i+1]
```

```
if random.number() < crossover_rate:
    child1, child2 = crossover(p1, p2)
else:
    child1, child2 = p1, p2

mutate(child1, mutation_rate, gene_bounds)
mutate(child2, mutation_rate, gene_bounds)

next_generation.add(child1)
next_generation.add(child2)

population = next_generation

return best_individual, best_fitness
```

Pros: Find global optimum
Faster optimisation of best solution

Cons: High computation cost
Constant parameter tuning

Application: Neural network training
DNA sequencing

12/8/25

summary

(2) Particle
optim
behav
on
particl
best
the
water
to
pers
val
to
rot

(3) Ant
that
short
by
ants
on
depo
allo
real
high
run
At
as
so

Code:

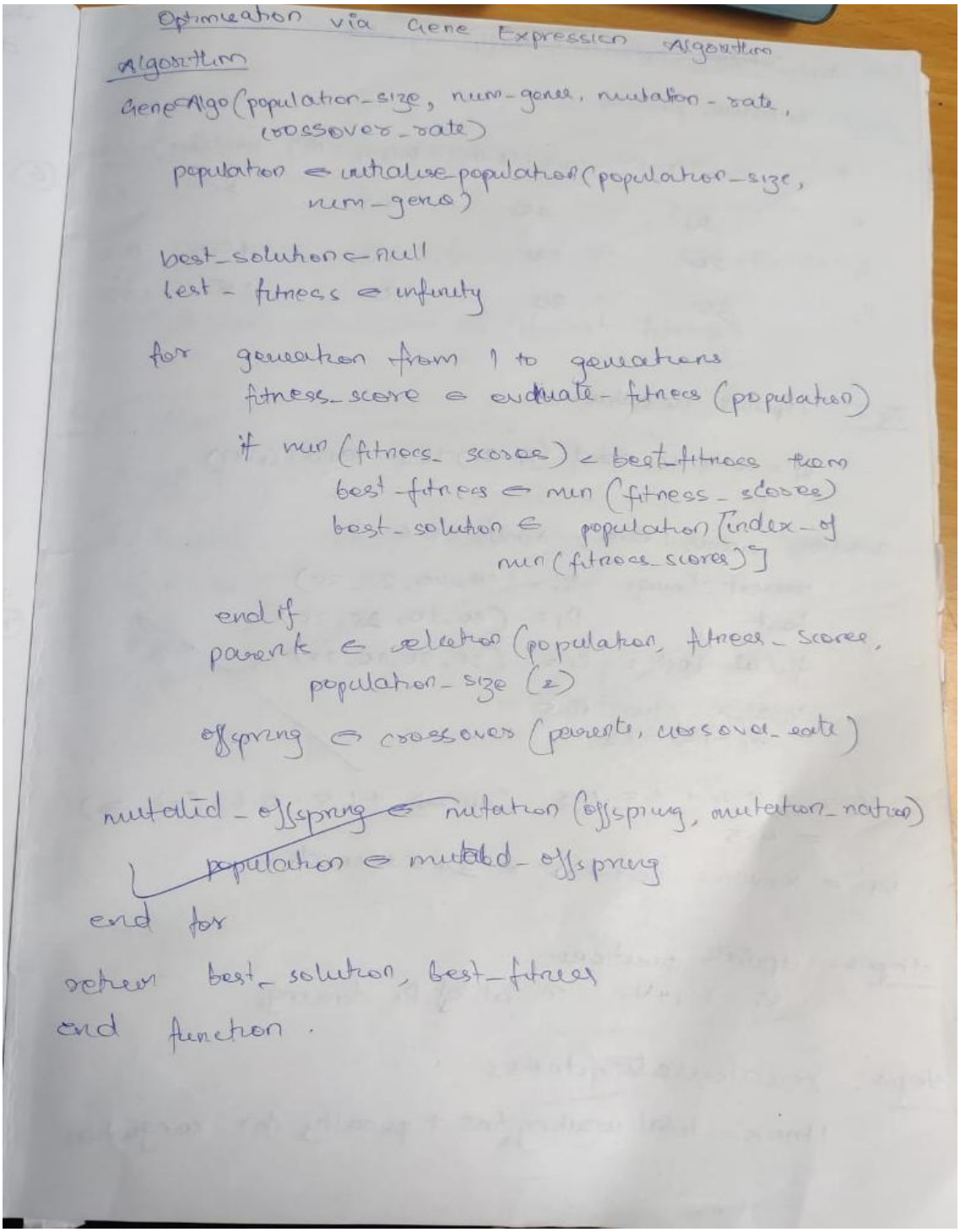
```
import random
import string

TARGET = "HELLO WORLD"
POP_SIZE = 100
MUTATION_RATE = 0.01
GENERATIONS = 1000
CHARS = string.ascii_uppercase + " "

def fitness(ind):
    return sum(c1 == c2 for c1, c2 in zip(ind, TARGET))

def generate_individual():
    return ''.join(random.choice(CHARS) for _ in TARGET)

def mutate(ind):
    return ''.join(
        c if random.random() > MUTATION_RATE else random.choice(CHARS)
        for c in ind
    )

def crossover(p1, p2):
    point = random.randint(0, len(TARGET) - 1)
    return p1[:point] + p2[point:]

def selection(pop):
    return max(random.sample(pop, 5), key=fitness)

def genetic_algorithm():
    population = [generate_individual() for _ in range(POP_SIZE)]

    for gen in range(GENERATIONS):
        population = sorted(population, key=fitness, reverse=True)
        best = population[0]
        print(f"Gen {gen}: {best} (Fitness = {fitness(best)})")
        if best == TARGET:
            break

        next_gen = population[:2]
        while len(next_gen) < POP_SIZE:
            p1 = selection(population)
            p2 = selection(population)
            child = crossover(p1, p2)
            child = mutate(child)
```

```
            next_gen.append(child)
        population = next_gen

    print("\nBest Match:", best)

if __name__ == "__main__":
    genetic_algorithm()
```

Program 2

Use a **gene expression** algorithm to find the optimal sequence for CPU process scheduling that minimizes the average waiting time of processes given their burst times.

Algorithm:



Optimization via Gene Expression Algorithm

Algorithm

```
GeneAlgo(population_size, num_genes, mutation_rate,
         crossover_rate)
    population ← initialise_population(population_size,
                 num_genes)

    best_solution ← null
    best_fitness ← infinity

    for generation from 1 to generations
        fitness_score ← evaluate_fitness(population)
        if min(fitness_scores) < best_fitness then
            best_fitness ← min(fitness_scores)
            best_solution ← population[index_of
                            min(fitness_scores)]

        end if
        parents ← selection(population, fitness_scores,
                  population_size/2)
        offspring ← crossover(parents, crossover_rate)

    mutated_offspring ← mutation(offspring, mutation_rate)
    population ← mutated_offspring
end for
return best_solution, best_fitness
end function.
```

Code:

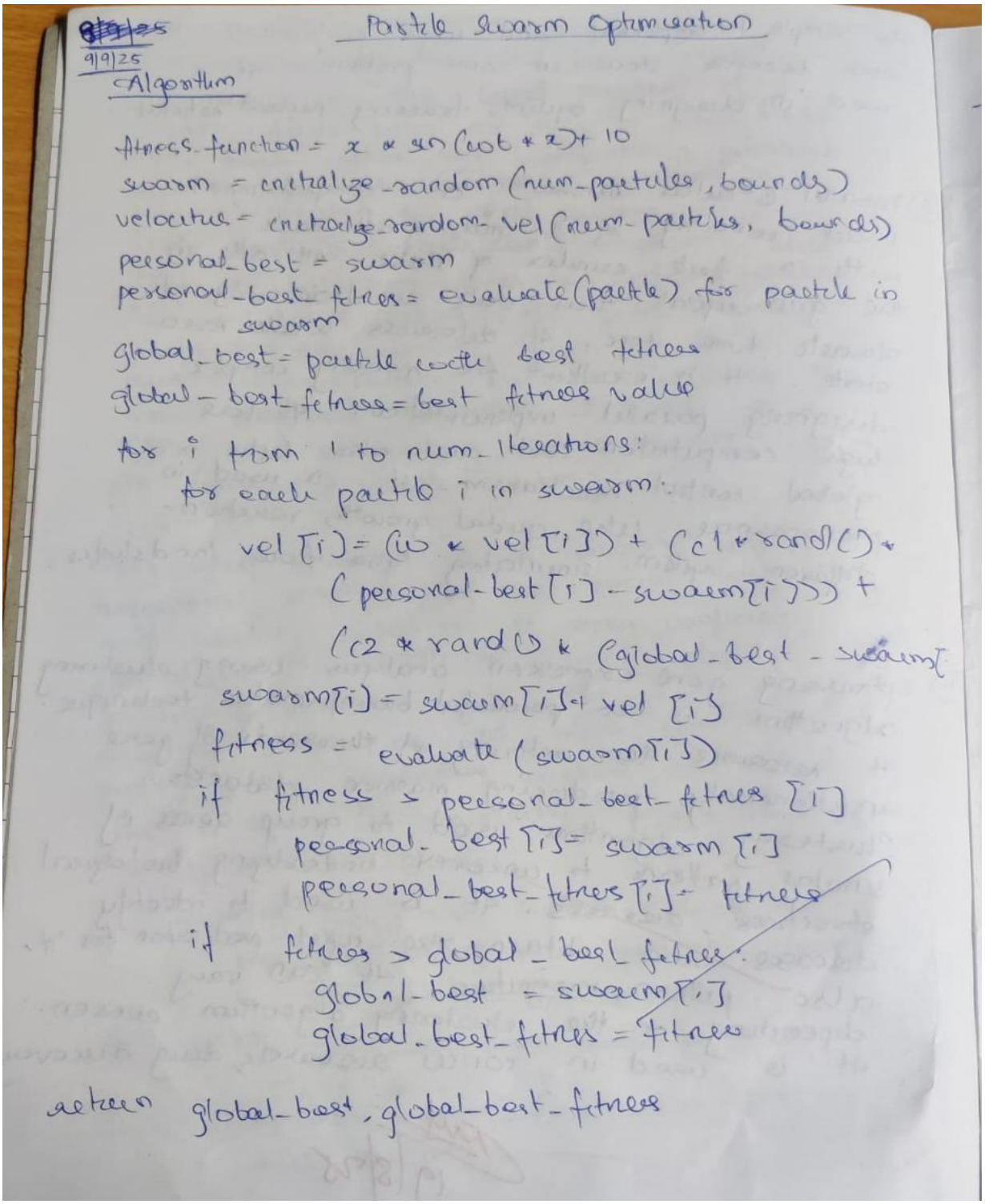
```
import random

# Calculate average waiting time for a given schedule
def avg_waiting_time(schedule, burst_times):
    waiting_time = 0
    total_waiting = 0
    for p in schedule[:-1]:
        total_waiting += waiting_time
        waiting_time += burst_times[p]
    return total_waiting / len(schedule)
# Fitness function (we want to minimize avg waiting time)
def fitness(schedule, burst_times):
    return 1 / (1 + avg_waiting_time(schedule, burst_times))

# Generate random schedule
def random_schedule(n):
    schedule = list(range(n))
    random.shuffle(schedule)
    return schedule

# Tournament selection
def selection(population, burst_times):
    contenders = random.sample(population, 3)
    return max(contenders, key=lambda s: fitness(s, burst_times))

# Crossover (ordered crossover for schedules)
def crossover(p1, p2):
    a, b = sorted(random.sample(range(len(p1)), 2))
    child = [None] * len(p1)
    child[a:b] = p1[a:b]
    fill = [p for p in p2 if p not in child]
    idx = 0
    for i in range(len(p1)):
        if child[i] is None:
            child[i] = fill[idx]
            idx += 1
    return child

# Mutation: swap two processes
def mutate(schedule, rate=0.2):
    for _ in range(len(schedule)):
        if random.random() < rate:
            i, j = random.sample(range(len(schedule)), 2)
            schedule[i], schedule[j] = schedule[j], schedule[i]
    return schedule

# Main GEA for CPU Scheduling
```

```
def gene_expression_scheduler(burst_times, pop_size=30, generations=200):
    n = len(burst_times)
    population = [random_schedule(n) for _ in range(pop_size)]
    best = None

    for g in range(generations + 1):  # include final generation
        population.sort(key=lambda s: fitness(s, burst_times), reverse=True)
        if best is None or fitness(population[0], burst_times) > fitness(best, burst_times):
            best = population[0]
        new_pop = []
        while len(new_pop) < pop_size:
            p1 = selection(population, burst_times)
            p2 = selection(population, burst_times)
            child = crossover(p1, p2)
            child   =   mutate(child)
            new_pop.append(child)

        population = new_pop

        # Print only selected generations
        if g == 0 or g == 10 or g % 50 == 0:
            print(f"Gen {g}: Best Avg Waiting Time = {avg_waiting_time(best, burst_times):.2f}")

    return best

# Example with 5 processes
burst_times = [5, 2, 8, 3, 6]  # CPU burst times for processes P1...P5
best_schedule = gene_expression_scheduler(burst_times)

print("\n Best Schedule Found:", best_schedule)
print(" Avg Waiting Time:", avg_waiting_time(best_schedule, burst_times))
```

Program 3

Find the maximum of the function f(x)=−x^2+20x+5 using **Particle Swarm Optimization** by iteratively updating particle positions and velocities.

Algorithm:



Particle Swarm Optimization

9/9/25

Algorithm

fitness_function = x * sin(w * x) + 10

swarm = initialize_random(num_particles, bounds)

velocities = initialize_random_vel(num_particles, bounds)

personal_best = swarm

personal_best_fitness = evaluate(particle) for particle in swarm

global_best = particle with best fitness

global_best_fitness = best fitness value

for i from 1 to num_iterations:

for each particle i in swarm:

vel[i] = (w * vel[i]) + (c1 * rand() * (personal_best[i] - swarm[i])) +

(c2 * rand() * (global_best - swarm[

swarm[i] = swarm[i] + vel[i]

fitness = evaluate(swarm[i])

if fitness > personal_best_fitness[i]

personal_best[i] = swarm[i]

personal_best_fitness[i] = fitness

if fitness > global_best_fitness:

global_best = swarm[i]

global_best_fitness = fitness

return global_best, global_best_fitness

⇒ Application (optimizing traffic signal)

step 1: initialize particles.

P	lane 1 (N-S)	lane 2 (E-W)	lane 3 (S-N)	lane 4 (W-E)	Total time
P_1	30	40	25	35	130
P_2	20	50	30	40	140
P_3	35	30	40	25	130

step 2: update velocities

$$V_i = w \cdot V_i + c_1 \cdot rand_1 \cdot (P_i - x_i) + c_2 \cdot rand_2 (G_i - x_i)$$

assume $rand_1 = 0.8$, $rand_2 = 0.6$

current timings: $X_1 = (30, 40, 25, 35)$

best $P_1 = (30, 40, 25, 35)$

global best: $G_1 = (35, 30, 40, 25)$

inertia: $w = 0.5$

$c_1 = 1.5 = c_2$

$$V_1 = 0.5 V_1 + 1.5 \cdot 0.8 \cdot (30 - 30) + 1.5 \cdot 0.6 \cdot (35 - 30)$$

$$= 4.5$$

$$x_1 = x_1 + V_1 = 30 + 4.5 = 34.5 \text{ seconds.}$$

step 3: update positions.

$x_1 = x_1 + V_1$ on all of P_1 timings.

step 4: recalculate fitness

fitness = total waiting time + penalty for congestion

After 1 iteration

Particle	X_i	V_i	$f(X_i)$	P_i
P1	(34.5, 40, 25, 35)	(4.5, 0, 0, 0)	120	(34.5, 40, 25, 35)
P2	(22, 50, 30, 40)	(2, 0, 0, 0)	125	(22, 50, 30, 40)
P3	(35, 31, 40, 25)	(0, 1, 0, 0)	118	(35, 31, 40, 25)

global best G = P3 ⇒ 118 (lowest fitness)

Step 5 Repeat till convergence

Result ⇒

Particle	Lane 1	lane 2	lane 3	lane 4	total time	fitness
P1 (best)	34	28	36	32	130	105

9/9/25

Code:

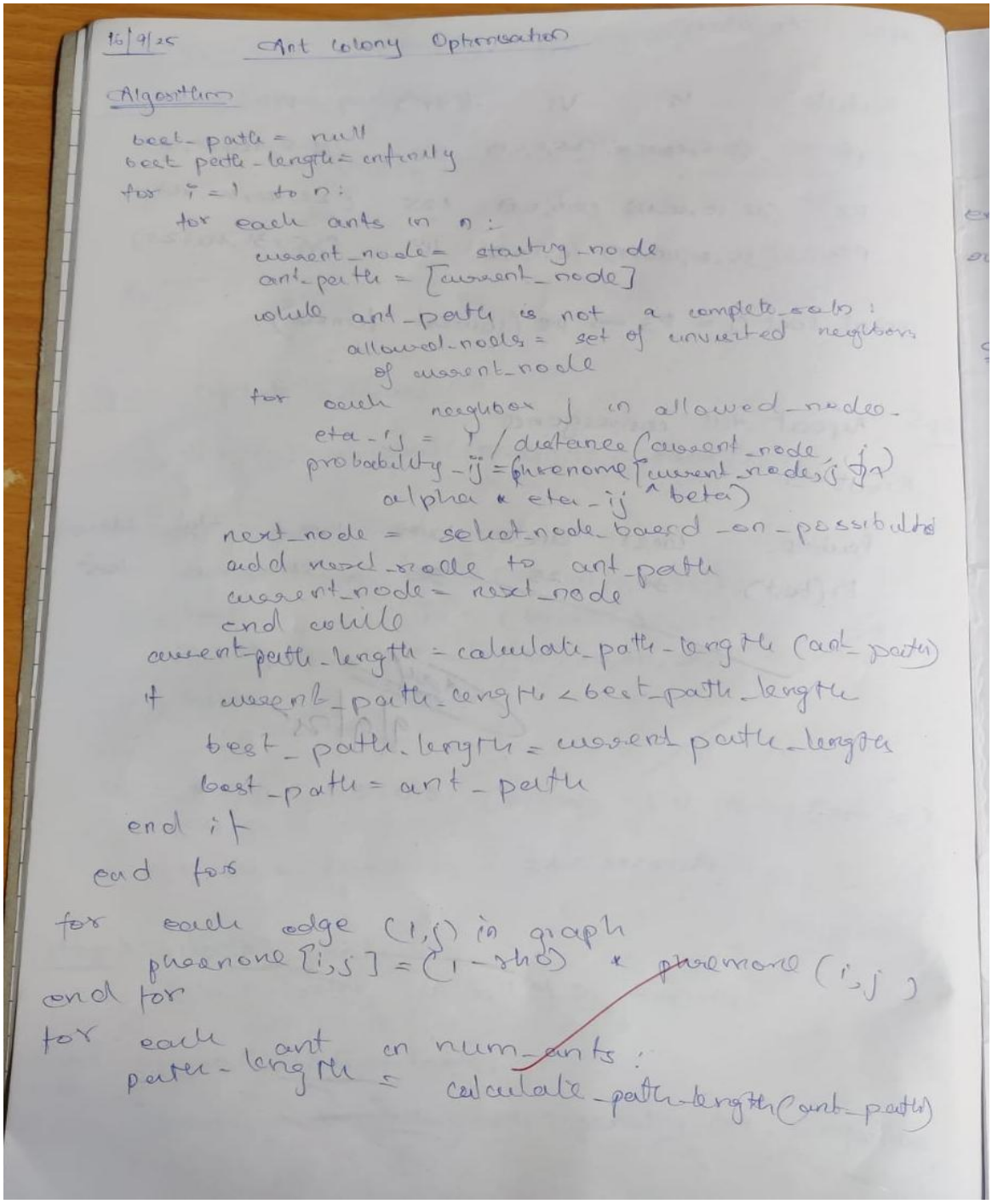
```
import numpy as np

# Objective function
def f(x):
    return -x**2 + 20*x + 5

# Initial particle positions
positions = np.array([0.6, 2.3, 2.8, 8.3, 10, 9.6, 6, 2.6, 1.1])
velocities = np.zeros_like(positions)
pbest_positions = positions.copy()
pbest_scores = f(positions)
gbest_position = pbest_positions[np.argmax(pbest_scores)]
c1 = c2 = 1
w = 1
# Random values from the example for each iteration
r_values = [
    (0.213, 0.876),
    (0.113, 0.706),
    (0.178, 0.507)
]

print("Initial positions:\n", positions)
print("Initial function values:\n", f(positions))
print("Initial global best position:", gbest_position, "with value:", f(gbest_position))
print("-" * 50)

# Run 3 iterations as in the example
for t in range(3):
    r1, r2 = r_values[t]

    for i in range(len(positions)):
        velocities[i] = (
            w * velocities[i] +
            c1 * r1 * (pbest_positions[i] - positions[i]) +
            c2 * r2 * (gbest_position - positions[i])
        )

    positions += velocities
    scores = f(positions)

    # Update personal bests
    for i in range(len(positions)):
        if scores[i] > pbest_scores[i]:
            pbest_positions[i] = positions[i]
```

```
        pbest_scores[i] = scores[i]

# Update global best
gbest_position = pbest_positions[np.argmax(pbest_scores)]

# Display results
print(f"Iteration {t+1}")
print("Positions:", positions.round(4))
print("Velocities:", velocities.round(4))
print("Function values:", scores.round(4))
print("Global best position:", gbest_position, "with value:", f(gbest_position))
print("-" * 50)
```

Program 4

Find the shortest route visiting all cities once and returning to the start using **Ant Colony Optimization.**

Algorithm:



```
    delta-pheromone = Q / path_length // Q is a const.
    for each edge (i, j) in ant_path:
        pheromone[i,j] = pheromone[i,j] + delta-pheromone
    end for
  end for
end for
return best_path
```

Application (Task Scheduling problem)

1. Define the problem

Task	Processing Time
T1	5
T2	9
T3	7
T4	6

2. Initialize parameters:

pheromone matrix → start with uniform pheromone(1)

Task / Machine	M1	M2	M3
T1	1.0	1.0	1.0
T2	1.0	1.0	1.0
T3	1.0	1.0	1.0
T4	1.0	1.0	1.0

Heuristic matrix $\Rightarrow \eta_{i,m} = 1$

	M_1	M_2	M_3
T_1	1.0	1.0	1.0
T_2	1.0	1.0	1.0
T_3	1.0	1.0	1.0
T_4	1.0	1.0	1.0

no-of-ants = 5

$\alpha = 1$

$\beta = 2$

$\rho = 0.5$

Iteration = 20

3. iteration 1 by ant1

T_1	M_1
T_2	M_2
T_3	M_3
T_4	M_1

machine load $\Rightarrow$ M1: T1(5) + T4(6) = 11

M2 : T2(9) = 9

M3 : T3(7) = 7

thus max(11, 9, 7) = 11

4. makespan = 11 min is the max load.

5. updating pheromone $\Rightarrow$ $T_{i,m} \in \rho \cdot T_{i,m}$

	M_1	M_2	M_3
T1	0.5	0.5	0.5
T2	0.5	0.5	0.5
T3	0.5	0.5	0.5
T4	0.5	0.5	0.5

pheromone deposition $\Rightarrow \Delta\tau = \frac{1}{11} = 0.0909$

	M_1	M_2	M_3
T_1	0.5909	0.5	0.5
T_2	0.5	0.5909	0.5
T_3	0.5	0.5	0.5909
T_4	0.5909	0.5	0.5

6. Repeat till 20 iterations.

7. Final output $\Rightarrow$

best schedule is same as earlier in step 3 $\Rightarrow$

makespan = max (11, 9, 7) = 11 minutes.

Task	Machine
T_1	M_1
T_2	M_2
T_3	M_3
T_4	M_1

16/9

Code:

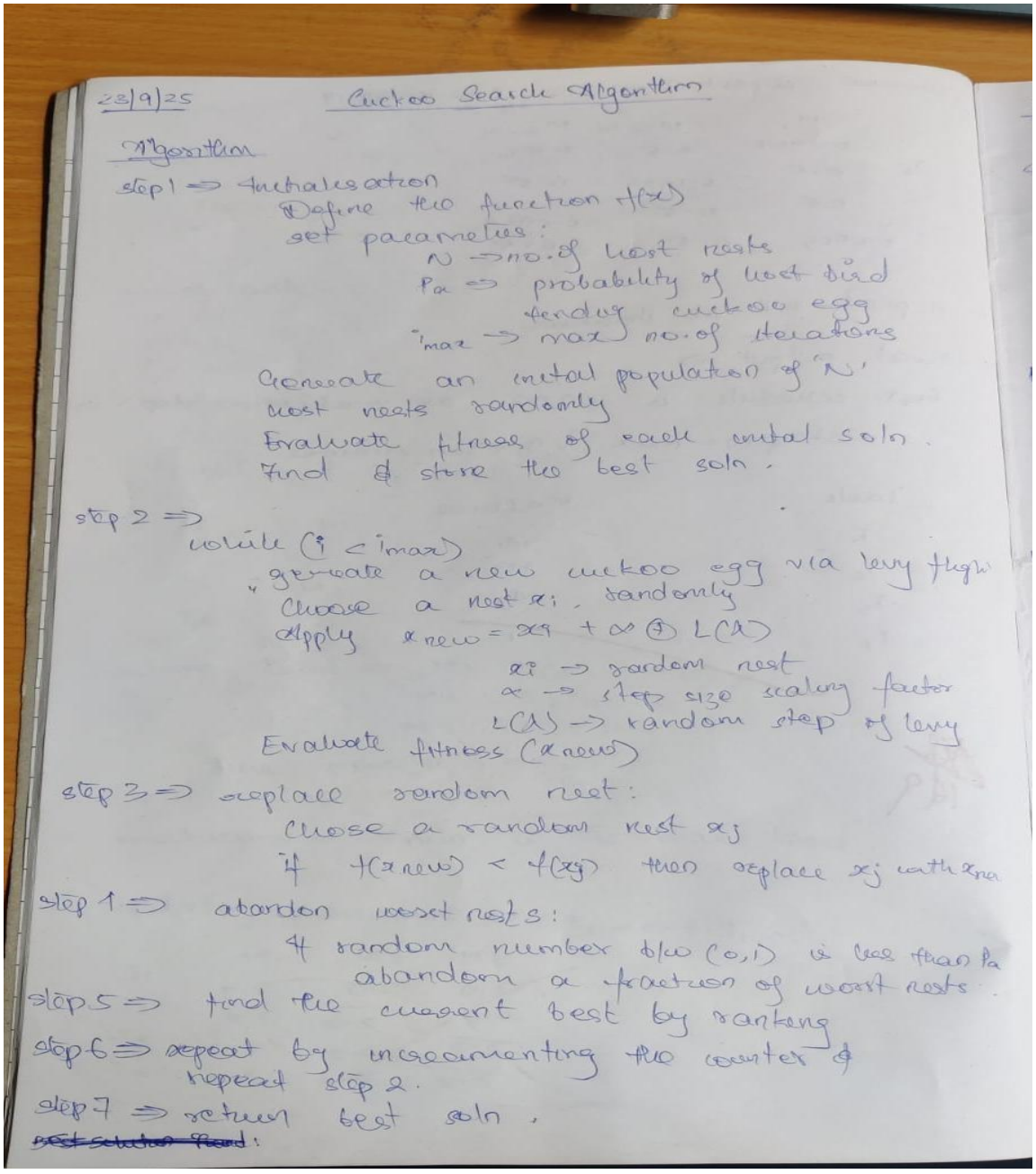
```
import numpy as np
import random
import math

# Step 1: Define the problem (set of cities with coordinates)
cities = [
    (0, 0),
    (1, 5),
    (5, 2),
    (6, 6),
    (8, 3),
    (7, 9),
    (2, 7),
    (3, 3)
]
N = len(cities)

# Calculate Euclidean distance matrix
def euclidean_distance(a, b):
    return math.sqrt((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2)

distance_matrix = [[euclidean_distance(cities[i], cities[j]) for j in range(N)] for i in range(N)]

# Step 2: Initialize Parameters
num_ants = N
max_iterations = 100
alpha = 1.0
beta = 5.0
rho = 0.5
Q = 100.0
tau_0 = 1.0

pheromone = [[tau_0 for _ in range(N)] for _ in range(N)]
heuristic = [[1 / distance_matrix[i][j] if i != j else 0 for j in range(N)] for i in range(N)]

best_tour = None
best_length = float('inf')

# Step 3, 4, 5: Iterate
for iteration in range(max_iterations):
    all_tours = []
    all_lengths = []

    for ant in range(num_ants):
```

```
        unvisited = list(range(N))
        current_city = random.choice(unvisited)
        tour = [current_city]
        unvisited.remove(current_city)

        while unvisited:
            probabilities = []
            for j in unvisited:
                tau = pheromone[current_city][j] ** alpha
                eta = heuristic[current_city][j] ** beta
                probabilities.append(tau * eta)
            total = sum(probabilities)
            probabilities = [p / total for p in probabilities]
            next_city = random.choices(unvisited, weights=probabilities, k=1)[0]
            tour.append(next_city)
            unvisited.remove(next_city)
            current_city = next_city
        # Return to starting city
        tour.append(tour[0])
        tour_length = sum(distance_matrix[tour[i]][tour[i+1]] for i in range(N))
        all_tours.append(tour)
        all_lengths.append(tour_length)

        if tour_length < best_length:
            best_length = tour_length
            best_tour = tour

    # Step 4: Update Pheromones
    # Evaporation
    for i in range(N):
        for j in range(N):
            pheromone[i][j] *= (1 - rho)

    # Deposit
    for tour, length in zip(all_tours, all_lengths):
        for i in range(N):
            a = tour[i]
            b = tour[i+1]
            pheromone[a][b] += Q / length
            pheromone[b][a] += Q / length  # because TSP is symmetric

# Step 6: Output the Best Solution
print("Best tour:", best_tour)
print("Best tour length:", round(best_length, 2))
```

Program 5

Solve the 0/1 Knapsack Problem by using **Cuckoo Search** to maximize total item value without exceeding the weight limit.

Algorithm:



lowest point in a hilly landscape)

Initialisation:

set up a population: Scatter 5 explorers randomly across the landscape & give a GPS to each one to record the position & elevation (fitness)

Identify the best solution with lowest elevation. Lets say explorer 1 at 500m is the best

Levy flight: Generating a new solution

select an explorer: randomly pick one of them to make a move. ~~eg) explo~~ eg) explorer 2

The explorer takes a levy flight to lets say a position of 450m

Replacement:

Lets explorer B at 450m be compared to another explorer lets say C at 600m.

Since 450m < 600m, B replaces C.

Abandonment:

let say P(a) = 0.25, you tell your explorers to randomly check if they should abandon their spot, if they say "yes" replace the person at highest elevation with someone else randomly chosen at random location. This is to expand the area of exploration.

23/9/25

Iteration:

Repeat the above step 2, 3, 4 & abandon the worst spots

finally you will reach the true lowest point of the landscape.

Code:

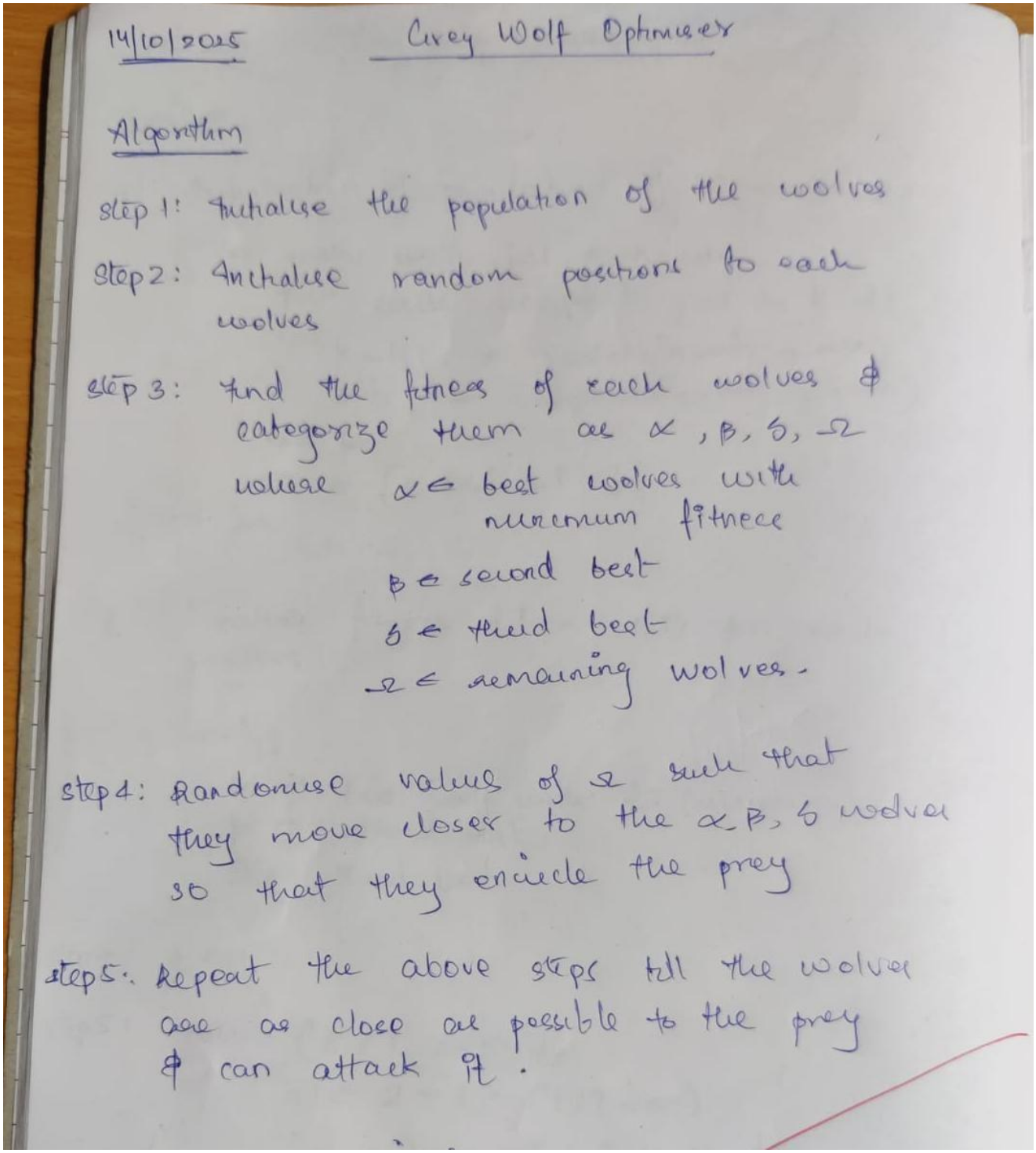
```
import random
import numpy as np

# Step 1: Define the Knapsack Problem
# Sample data: values and weights of items
values = [60, 100, 120, 80, 30]
weights = [10, 20, 30, 15, 5]
max_weight = 50
num_items = len(values)

# Fitness function: maximize value while staying under weight limit
def fitness(nest):
    total_value = 0
    total_weight = 0
    for i in range(num_items):
        if nest[i] == 1:
            total_value += values[i]
            total_weight += weights[i]
    if total_weight > max_weight:
        return 0  # Penalty for exceeding capacity
    return total_value

# Step 2: Initialize Parameters
num_nests = 25
max_iterations = 100
discovery_rate = 0.25  # Probability of discovering a nest
alpha = 1.0  # Step size for Lévy flights

# Step 3: Initialize Population
def initialize_population():
    return [[random.randint(0, 1) for _ in range(num_items)] for _ in range(num_nests)]

population = initialize_population()

# Step 4: Evaluate Fitness
fitness_values = [fitness(nest) for nest in population]

# Step 5: Generate New Solutions using Lévy flights (simple binary flipping)
def levy_flight(nest):
    new_nest = nest.copy()
    for i in range(num_items):
        if random.random() < 0.3:  # 30% chance to flip each bit
            new_nest[i] = 1 - new_nest[i]
    return new_nest
```

```
# Step 6: Replace worst nests
def abandon_worst_nests(population, fitness_values):
    num_abandon = int(discovery_rate * num_nests)
    worst_indices = np.argsort(fitness_values)[:num_abandon]

    for idx in worst_indices:
        population[idx] = [random.randint(0, 1) for _ in range(num_items)]
    return population

# Step 7: Main Loop
best_nest = None
best_fitness = -1

for iteration in range(max_iterations):
    for i in range(num_nests):
        # Generate a new solution by Lévy flight
        new_nest = levy_flight(population[i])
        new_fitness = fitness(new_nest)

        # Greedy selection
        if new_fitness > fitness_values[i]:
            population[i] = new_nest
            fitness_values[i] = new_fitness

        # Track the global best
        if new_fitness > best_fitness:
            best_fitness = new_fitness
            best_nest = new_nest.copy()

    # Abandon a fraction of the worst nests
    population = abandon_worst_nests(population, fitness_values)
    fitness_values = [fitness(nest) for nest in population]

# Step 8: Output the Best Solution
print("Best Solution (Item Inclusion):", best_nest)
print("Best Total Value:", best_fitness)
print("Total Weight:",
      sum(weights[i] for i in range(num_items) if best_nest[i] == 1))
```

Program 6

Optimize the assignment of cars to parking slots to minimize total walking distance using **Grey Wolf Optimization**, simulating the social hierarchy and hunting behavior of grey wolves to iteratively improve solutions.

Algorithm:



14/10/2025 Grey Wolf Optimiser

Algorithm

step 1: Initialise the population of the wolves

Step 2: Initialise random positions to each wolves

step 3: find the fitness of each wolves & categorize them as α, β, δ, Ω where $\alpha \in$ best wolves with minimum fitness

$\beta \in$ second best

$\delta \in$ third best

$\Omega \in$ remaining wolves.

step 4: Randomise values of Ω such that they move closer to the α, β, δ wolves so that they encircle the prey

step 5. Repeat the above steps till the wolves are as close as possible to the prey & can attack it.

Application (Robotic path planner)

goal → shortest path from start to target cell

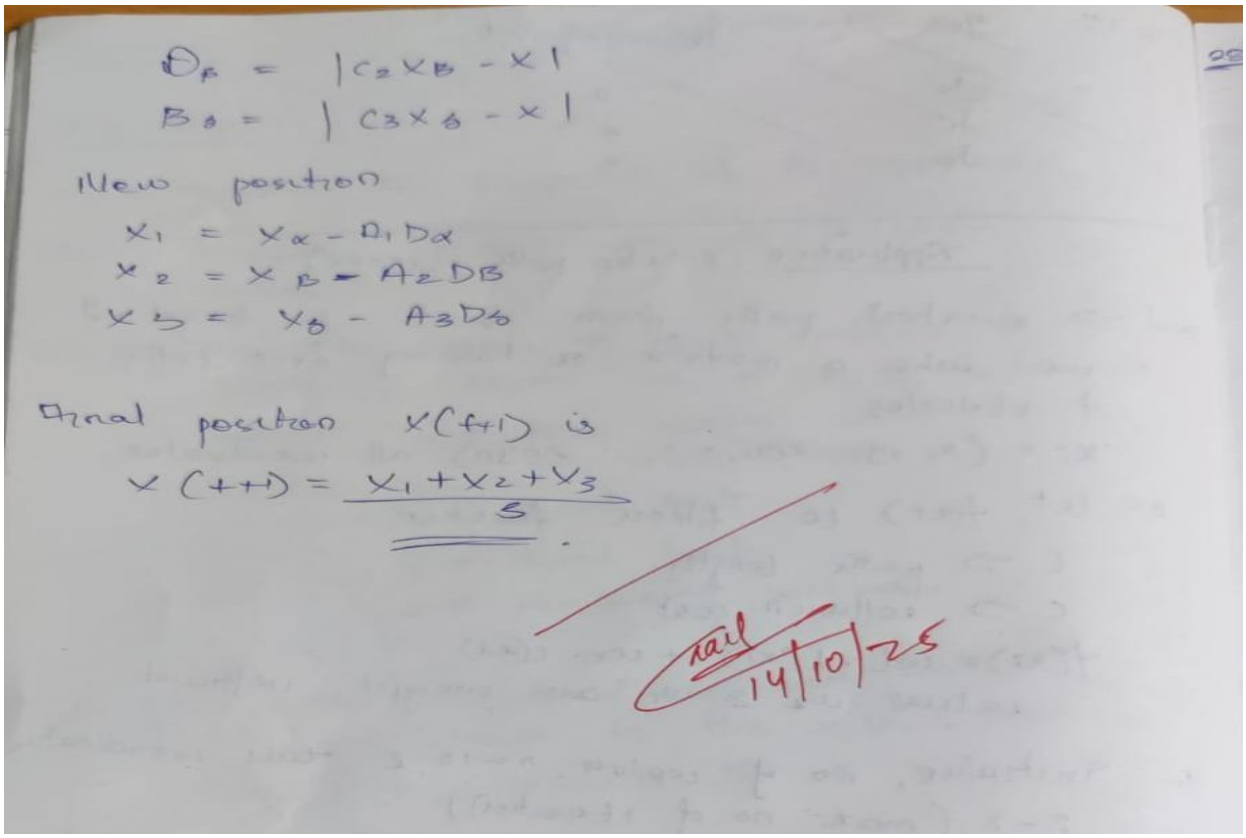
1. we take a matrix containing free cells & obstacles
   $X_i = (x_1, y_1, x_2 y_2, \cdots x_n y_n)$ all coordinates
2. let f(x) be fitness function.
   L → path length
   C → collision cost
   $f(x_i) = w_L \cdot L(x_i) + w_c \cdot c(x_i)$
   where w_L & w_c are weight coefficient
3. Initialise, no. of wolves, n = 10 & their coordinates
   i = 2 (max no. of iteration)
   calculate f(xi) for each wolf.
   α has lowest f(xi) followed by β, δ & ω)
4. encircling prey

Each ω wolf updates its position based on α, β, δ (3 best solution or paths)

Calculate distance D_α, D_β & D_δ

$D_\alpha = |C_1 X_\alpha - X|$

where X is distance of current wolf.



Code:

```
import numpy as np

# Example Data:
num_cars = 5
num_slots = 5
slot_distances = np.array([10, 20, 15, 30, 25]) # Distances of parking slots to entrance
num_wolves = 10
max_iter = 50

# Initialize wolves: each wolf is a permutation of slot indices representing car assignments
def initialize_population(num_wolves, num_slots):
    population = []
    for _ in range(num_wolves):
        wolf = np.random.permutation(num_slots)
        population.append(wolf)
    return population

# Fitness function: total walking distance for assigned slots
def fitness(wolf):
    total_distance = 0
    for car_index, slot_index in enumerate(wolf):
        total_distance += slot_distances[slot_index] # Distance of car's assigned slot
```

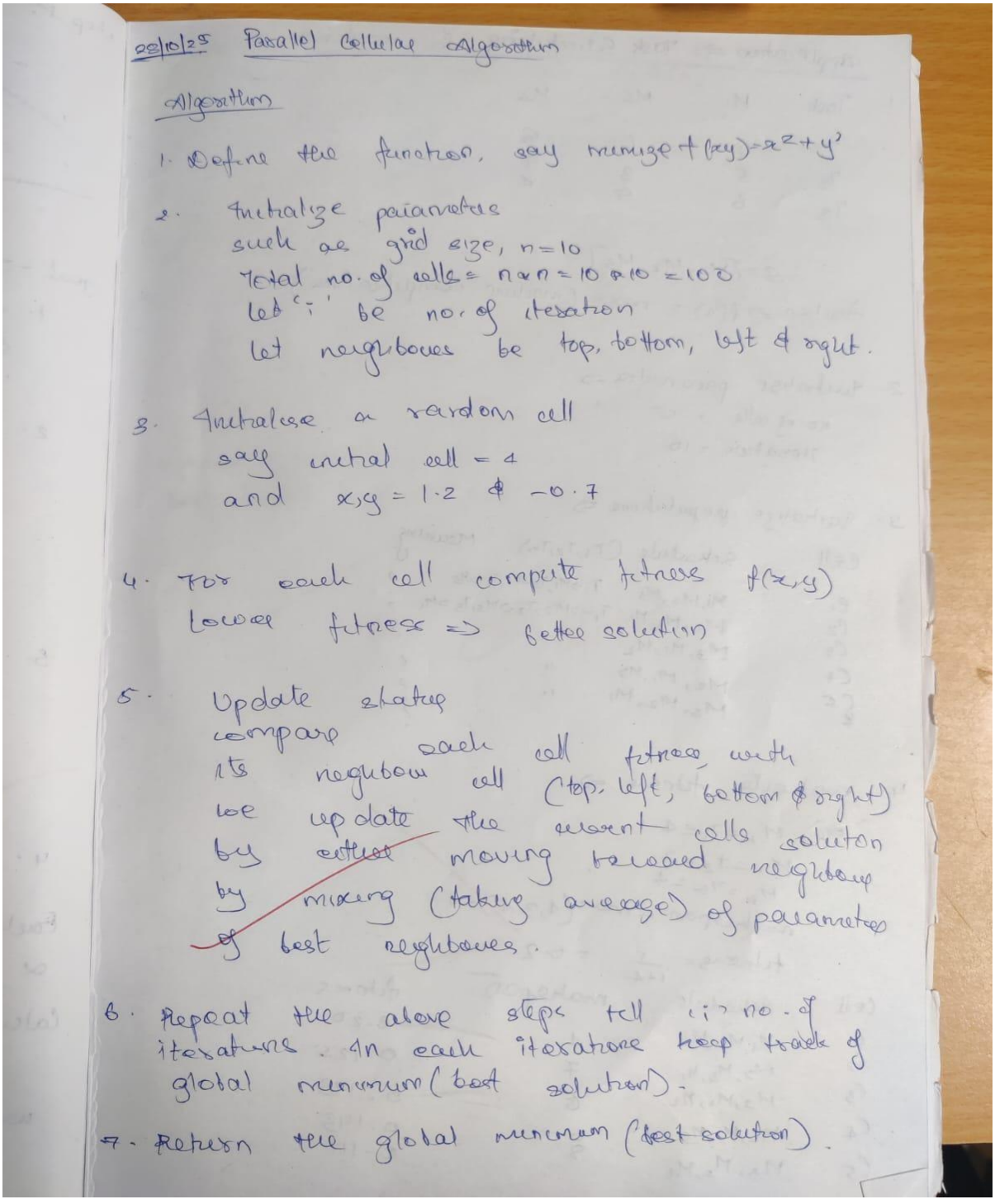
```
    return total_distance

# Update position of wolf based on alpha, beta, delta (GWO update mechanism)
def update_position(wolf, alpha, beta, delta, a):
    # Generate a new wolf position by exploring around alpha, beta, delta wolves
    new_wolf = wolf.copy()

    for i in range(len(wolf)):
        r1 = np.random.rand()
        r2 = np.random.rand()

        # Move the wolf toward the alpha, beta, or delta
        if r1 < 0.33:
            new_wolf[i] = alpha[i]  # Move toward alpha wolf
        elif r1 < 0.66:
            new_wolf[i] = beta[i]  # Move toward beta wolf
        else:
            new_wolf[i] = delta[i]  # Move toward delta wolf

    # Ensure new_wolf is a valid permutation (no duplicates)
    new_wolf = np.unique(new_wolf)
    if len(new_wolf) < len(wolf):
        # if duplicates occur, randomly swap elements to maintain uniqueness
        missing_values = list(set(range(len(wolf))) - set(new_wolf))
        np.random.shuffle(missing_values)
        new_wolf = np.append(new_wolf, missing_values)

    # Now we randomly swap some elements to increase diversity and exploration
    swap_indices = np.random.choice(len(wolf), 2, replace=False)
    new_wolf[swap_indices[0]], new_wolf[swap_indices[1]] = new_wolf[swap_indices[1]],
new_wolf[swap_indices[0]

    return new_wolf

# Main GWO loop for parking slot allocation
def gwo_parking_allocation():
    # Initialize wolves (population)
    population = initialize_population(num_wolves, num_slots)
    # Variables to keep track of the best solutions
    alpha = None
    beta = None
    delta = None

    alpha_score = float('inf')
    beta_score = float('inf')
    delta_score = float('inf')
    # GWO main loop (iterations)
```

```
    for iteration in range(max_iter):
        # Evaluate the fitness of each wolf in the population
        fitness_scores = []
        for wolf in population:
            score = fitness(wolf)
            fitness_scores.append(score)

            # Update alpha, beta, delta based on fitness
            if score < alpha_score:
                delta_score, delta = beta_score, beta
                beta_score, beta = alpha_score, alpha
                alpha_score, alpha = score, wolf
            elif score < beta_score:
                delta_score, delta = beta_score, beta
                beta_score, beta = score, wolf
            elif score < delta_score:
                delta_score, delta = score, wolf
        # Update positions of wolves based on best wolves (alpha, beta, delta)
        new_population = []
        for wolf in population:

            # Adjust the exploration factor "a" to control the step size over iterations
            a = 2 - iteration * (2 / max_iter)
            new_wolf = update_position(wolf, alpha, beta, delta, a)
            new_population.append(new_wolf)
        population = new_population

        # Print progress
        print(f"Iteration {iteration+1}: Best total walking distance = {alpha_score}")

    # Final best result
    print("\nBest Assignment of Cars to Slots (car i -> slot number):")
    print(alpha)
    print("Slot distances:", slot_distances[alpha])
    print("Total walking distance:", alpha_score)

# Run the optimizer
gwo_parking_allocation()
```

Program 7

Optimize the search efficiency of multiple drones exploring a grid by coordinating their positions based on neighbors' success using a **Parallel Cellular Algorithm** to maximize the total searched area.

Algorithm:



29/10/25 Parallel Cellular Algorithm

Algorithm

1. Define the function, say minimize $f(x,y) = x^2 + y^2$
2. Initialize parameters such as grid size, $n = 10$
   Total no. of cells $= n \times n = 10 \times 10 = 100$
   Let 'i' be no. of iteration
   Let neighbours be top, bottom, left & right.
3. Initialise a random cell
   say initial cell = 4
   and $x, y = 1.2$ & -0.7
4. For each cell compute fitness $f(x,y)$
   Lower fitness => better solution
5. Update status
   compare each cell fitness with its neighbour cell (top, left, bottom & right)
   be update the current cells soluton
   by either moving towards neighbour
   by mixing (taking average) of parameters of best neighbours.
6. Repeat the above steps till i -> no. of iterations. In each iteration keep tracks of global minimum (best solution).
7. Return the global minimum (best solution).

Application ⇒ Task scheduling for machines

1.

Task	M_1	M_2	M_3
T_1	4	6	8
T_2	5	3	7
T_3	6	5	4

$S = [M_1, M_2, M_3]$

fitness ⇒ $f(S)$ = max (machine completion times).

2. Initialise parameters ⇒

no. of cells = 5

iteration = 10

3. Initialize populations ⇒

Cell	schedule (T_1, T_2, T_3)	Meaning
C_1	M_1, M_2, M_3	$T_1 \to M_1, T_2 \to M_2, T_3 \to M_3$
C_2	M_2, M_3, M_1	$T_1 \to M_2, T_2 \to M_3, T_3 \to M_1$
C_3	M_3, M_1, M_2	"
C_4	M_2, M_1, M_3	"
C_5	M_3, M_2, M_1	"

4. evaluate fitness ⇒

eg) $M_1 = T_1 = 4$

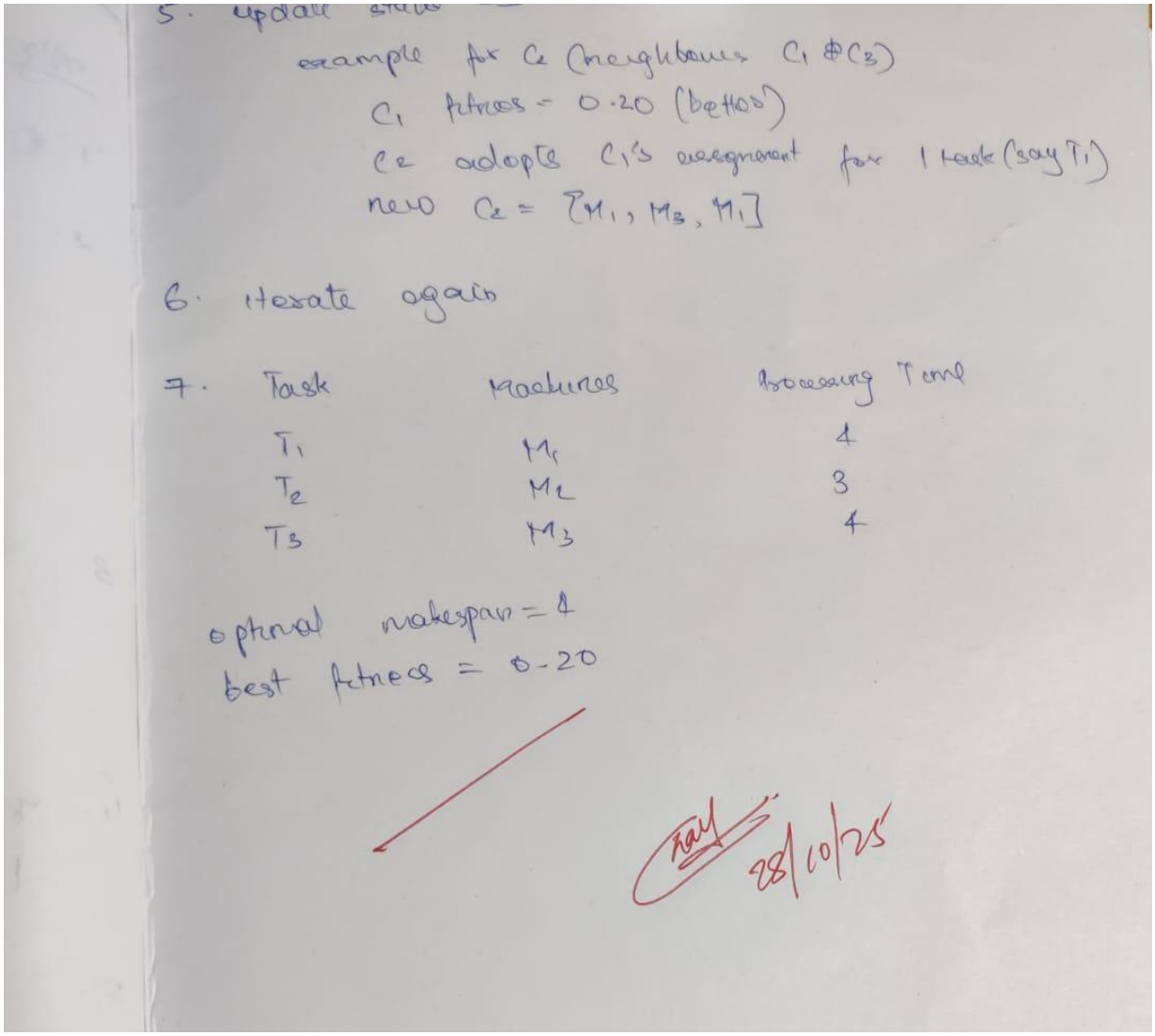
$M_2 = T_2 = 3$

$M_3 = T_3 = 4$

makespan = max(4, 3, 4) = 4

fitness = $\frac{1}{1+4} = 0.2$

Cell	schedule	makespan	fitness
C_1	M_1, M_2, M_3	4	0.20
C_2	M_2, M_3, M_1	7	0.125
C_3	M_3, M_1, M_2	6	0.143
C_4	M_2, M_1, M_3	6	0.143
C_5	M_3, M_2, M_1	9	0.111



Code:

```
import random
import numpy as np
# Define the grid size (search area)
GRID_SIZE = 5  # 5x5 grid
NUM_DRONES = 10  # Number of drones

# Define the drone class to represent each drone
class Drone:
    def __init__(self, x, y):
        self.x = x  # x-coordinate of the drone
        self.y = y  # y-coordinate of the drone
        self.fitness = 0  # Fitness: how much area it has searched successfully
```

```
    def search_area(self):
        # Simulate the drone searching its area (random chance of finding something)
        if random.random() < 0.2:  # 20% chance the drone finds something
            self.fitness += 1
        return self.fitness

    def update_position(self, new_x, new_y):
        # Update drone position
        self.x = new_x
        self.y = new_y

# Function to initialize drones in random positions within the grid
def initialize_drones():
    drones = []
    for _ in range(NUM_DRONES):
        x = random.randint(0, GRID_SIZE - 1)
        y = random.randint(0, GRID_SIZE - 1)
        drones.append(Drone(x, y))
    return drones

# Function to evaluate the fitness of all drones (i.e., how much area they've searched)
def evaluate_drones(drones):
    total_fitness = 0
    for drone in drones:
        total_fitness += drone.search_area()
    return total_fitness

# Function to get the neighboring drones of a given drone
def get_neighbors(drone, drones):
    neighbors = []
    for other_drone in drones:
        if other_drone != drone:
            # Check if within 1 distance in either x or y direction
            if abs(drone.x - other_drone.x) <= 1 and abs(drone.y - other_drone.y) <= 1:
                neighbors.append(other_drone)
    return neighbors
# Function to update drone's position based on neighbors' positions
def update_drone_position(drone, neighbors):
    # Simple strategy: move to a nearby unexplored area if possible
    if neighbors:
        # Get the best neighbor (one with the highest fitness)
        best_neighbor = max(neighbors, key=lambda n: n.fitness)
        # Move towards the best neighbor's position (if it's not already there)
        if best_neighbor.x != drone.x or best_neighbor.y != drone.y:
            drone.update_position(best_neighbor.x,  best_neighbor.y)

# Main function to simulate the drone search and rescue operation
```

```
def search_and_rescue():
    drones = initialize_drones()
    best_fitness = 0

    # Run for a set number of iterations (time cycles)
    num_iterations = 20
    for iteration in range(num_iterations):
        # Evaluate and update the drones' fitness
        total_fitness = evaluate_drones(drones)

        # Track the best fitness (maximize search efficiency)
        if total_fitness > best_fitness:
            best_fitness = total_fitness
            print(f"Iteration {iteration + 1}: Best Fitness = {best_fitness}")

        # Update the drones' positions based on neighbors
        for drone in drones:
            neighbors = get_neighbors(drone, drones)
            update_drone_position(drone, neighbors)

    return best_fitness

# Run the simulation
best_fitness = search_and_rescue()
print(f"Best Fitness Achieved: {best_fitness}")
```